

บทที่ 4 แพตเทิร์นโครงสร้าง (Structural Patterns)

4.1 ความสำคัญของแพตเทิร์นโครงสร้าง

แพตเทิร์นโครงสร้าง (Structural Patterns) เป็นกลุ่มแพตเทิร์นที่ช่วยแก้ปัญหาเกี่ยวกับโครงสร้างของวัตถุ มีจุดประสงค์หลักเพื่อช่วยให้นักพัฒนาออกแบบโครงสร้างขององค์ประกอบคลาสภายในระบบซอฟต์แวร์ที่มีประสิทธิภาพ และสามารถขยายได้ง่าย โดยเน้นเรื่องความสัมพันธ์ระหว่างคลาสหรืออ็อบเจกต์

เหตุผลที่แพตเทิร์นโครงสร้างมีความสำคัญ

- 1) **ลดความซับซ้อน** ช่วยให้องค์ประกอบในระบบสามารถเชื่อมต่อ และทำงานร่วมกันได้ สามารถจัดการความสัมพันธ์ระหว่างคลาสและอ็อบเจกต์ได้อย่างมีประสิทธิภาพ เช่น การใช้ Adapter Pattern เพื่อให้คลาสที่แตกต่างกันสามารถทำงานร่วมกันได้
- 2) **สนับสนุนหลักการเชิงวัตถุ** โดยใช้ Encapsulation, Abstraction, Inheritance และ Composition เพื่อออกแบบโครงสร้างที่ชัดเจนและยืดหยุ่น ช่วยลดการพึ่งพา (Dependency) ระหว่างส่วนต่าง ๆ ของระบบ
- 3) **ส่งเสริมการนำโค้ดกลับมาใช้ซ้ำ** (Reusability) ช่วยให้สามารถสร้างองค์ประกอบที่สามารถนำกลับมาใช้ซ้ำในส่วนอื่นของโปรแกรมได้ เช่น การใช้ Composite Pattern สำหรับการจัดการโครงสร้างแบบลำดับชั้น (Hierarchy)
- 4) **รองรับการขยาย** (Scalability) ระบบที่ออกแบบด้วยแพตเทิร์นโครงสร้างสามารถรองรับการเพิ่มฟีเจอร์ใหม่ได้ง่าย เช่น การใช้ Decorator Pattern เพื่อเพิ่มฟังก์ชันใหม่ให้กับอ็อบเจกต์โดยไม่ต้องแก้ไขโค้ดเดิม

แพตเทิร์นโครงสร้าง มีทั้งหมด 7 แพตเทิร์น ดังรายละเอียดในหัวข้อต่อไป

4.2 Adapter Pattern

Adapter Design Pattern มีหน้าที่ทำให้คลาสที่มีอินเทอร์เฟซแตกต่างกันสามารถทำงานร่วมกันได้ ด้วยการสร้าง Adapter class เพื่อแปลงอินเทอร์เฟซของคลาสหนึ่งให้เข้ากับอีกคลาสหนึ่ง

:: *Adapter Pattern* ::

ทำให้คลาสที่มีอินเทอร์เฟซแตกต่างกันสามารถทำงานร่วมกันได้

โดยสร้าง Adapter class เพื่อแปลงอินเทอร์เฟซของคลาสหนึ่งให้เข้ากับอีกคลาสหนึ่ง

ตัวอย่างปัญหา

ระบบบ้านอัจฉริยะ (Smart Home) ที่ควบคุมอุปกรณ์ไฟฟ้าผ่านอินเทอร์เฟซมาตรฐาน (SmartDevice) แต่ในระบบมีอุปกรณ์รุ่นเก่า (Legacy) ที่ไม่รองรับอินเทอร์เฟซนี้

ตัวอย่าง Main program

```
SmartDevice legacyLight = new LegacyLightAdapter(  
                                new LegacyLight()  
                                );  
legacyLight.turnOn();
```

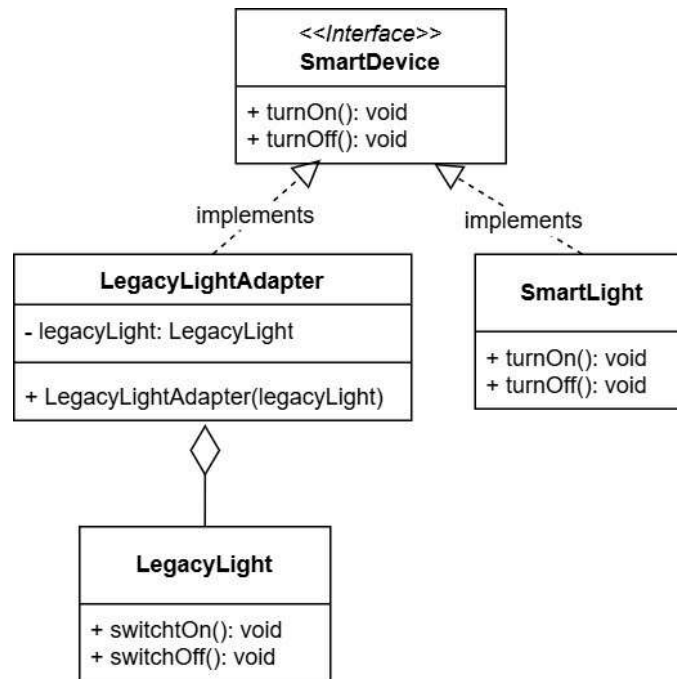
4.2.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Adapter Design Pattern

- รวมระบบที่แตกต่างกัน (System Integration) เมื่อมีสองระบบหรือโมดูลที่ออกแบบแยกจากกัน แต่จำเป็นต้องทำงานร่วมกัน เช่น การเชื่อมต่อบ้านอัจฉริยะกับระบบเก่าหรือ API ที่มีอยู่แล้ว
- ปรับเปลี่ยนอินเทอร์เฟซของคลาสเดิม แทนที่จะต้องแก้ไขโค้ดทั้งหมดของระบบ
- ช่วยให้สามารถเพิ่มฟังก์ชันใหม่ในระบบได้ โดยไม่ต้องเปลี่ยนโครงสร้างของคลาสที่มีอยู่แล้ว

ตัวอย่างการใช้งาน

สมมติว่าระบบบ้านอัจฉริยะ (Smart Home) ที่ควบคุมอุปกรณ์ไฟฟ้าผ่านอินเทอร์เน็ตมาตรฐาน (SmartDevice) แต่ในระบบมีอุปกรณ์รุ่นเก่าที่ไม่รองรับอินเทอร์เน็ตนี้ เช่น หลอดไฟธรรมดา (LegacyLight) ซึ่งมีวิธีการเปิดปิดที่แตกต่างกัน



ภาพที่ xx

คลาส SmartDevice อินเทอร์เน็ตมาตรฐานสำหรับอุปกรณ์อัจฉริยะ

```
1. // Step 1: อินเทอร์เน็ตมาตรฐานสำหรับอุปกรณ์อัจฉริยะ
2. interface SmartDevice {
3.     void turnOn();
4.     void turnOff();
5. }
```

คลาส SmartLight อุปกรณ์อัจฉริยะที่รองรับอินเทอร์เน็ตมาตรฐานโดยตรง

```
1. // Step 2: อุปกรณ์อัจฉริยะที่รองรับอินเทอร์เน็ต
2. class SmartLight implements SmartDevice {
3.
4.     @Override
5.     public void turnOn() {
6.         System.out.println("Smart Light is turned ON.");
7.     }
8.
9.     @Override
```

```

10.     public void turnOff() {
11.         System.out.println("Smart Light is turned OFF.");
12.     }
13.}

```

คลาส LegacyLight อุปกรณ์รุ่นเก่าที่มีอินเทอร์เฟซไม่ตรงกับมาตรฐานสำหรับอุปกรณ์อัจฉริยะ

```

1. // Step 3: อุปกรณ์รุ่นเก่าที่มีอินเทอร์เฟซแตกต่าง
2. class LegacyLight {
3.
4.     public void switchOn() {
5.         System.out.println("Legacy Light is switched ON.");
6.     }
7.
8.     public void switchOff() {
9.         System.out.println("Legacy Light is switched OFF.");
10.    }
11.}

```

คลาส LegacyLightAdapter ตัวแปลง (Adapter) ที่แปลงการทำงานของ LegacyLight ให้เข้ากับ SmartDevice

```

1. // Step 4: สร้าง Adapter เพื่อแปลง LegacyLight ให้รองรับ SmartDevice
2. class LegacyLightAdapter implements SmartDevice {
3.
4.     private LegacyLight legacyLight;
5.
6.     public LegacyLightAdapter(LegacyLight legacyLight) {
7.         this.legacyLight = legacyLight;
8.     }
9.
10.    @Override
11.    public void turnOn() {
12.        legacyLight.switchOn(); // แปลงการเรียกใช้งาน
13.    }
14.
15.    @Override
16.    public void turnOff() {
17.        legacyLight.switchOff(); // แปลงการเรียกใช้งาน
18.    }
19.}

```

คลาส Main ใช้งานทั้งอุปกรณ์อัจฉริยะและอุปกรณ์รุ่นเก่าผ่านอินเทอร์เฟซเดียวกัน

```

1. // Step 5: การใช้งานในระบบบ้านอัจฉริยะ
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // อุปกรณ์อัจฉริยะ
6.         SmartDevice smartLight = new SmartLight();
7.         smartLight.turnOn();
8.         smartLight.turnOff();
9.

```

```

10.          // อุปกรณ์รุ่นเก่าผ่าน Adapter
11.          SmartDevice legacyLight = new LegacyLightAdapter(
12.                                  new LegacyLight());
13.          legacyLight.turnOn();
14.          legacyLight.turnOff();
15.      }
16.}

```

ผลลัพธ์ของโปรแกรม

Smart Light is turned ON.

Smart Light is turned OFF.

Legacy Light is switched ON.

Legacy Light is switched OFF.

4.2.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Adapter Pattern เหมาะสำหรับสถานการณ์ที่ต้องเชื่อมต่อหรือรวมระบบที่มีอินเทอร์เฟซไม่ตรงกัน แต่ต้องการทำงานร่วมกัน ปัญหาในงานจริงที่สามารถแก้ไขได้ด้วย Adapter Pattern มีดังนี้

ตัวอย่างที่ 1 การเชื่อมต่อระบบเก่ากับระบบใหม่ บริษัทมีระบบเก่าที่ใช้มานาน และมีการพัฒนา API ใหม่เพื่อรองรับการทำงานในระบบใหม่ แต่ระบบเก่าและ API ใหม่มีโครงสร้างและอินเทอร์เฟซที่แตกต่างกัน ทำให้ไม่สามารถเชื่อมต่อกันได้โดยตรง สามารถใช้ Adapter Pattern เพื่อแปลงอินเทอร์เฟซของระบบเก่าให้เข้ากับ API ใหม่

```

1. // ระบบใหม่ (Target Interface)
2. interface NewPaymentGateway {
3.     void processPayment(double amount);

```

```

1. // ระบบเก่า (Adaptee)
2. class OldPaymentSystem {
3.     public void makePayment(String amount) {
4.         System.out.println("Payment of " + amount
5.             + " processed using Old Payment System.");
6.     }
7. }

```

```

1. // Adapter สำหรับระบบเก่า
2. class PaymentAdapter implements NewPaymentGateway {
3.
4.     private OldPaymentSystem oldSystem;
5.
6.     public PaymentAdapter(OldPaymentSystem oldSystem) {
7.         this.oldSystem = oldSystem;

```

```

12.     }
13.
14.     @Override
15.     public void processPayment(double amount) {
16.         // แปลงข้อมูลและเรียกใช้งานระบบเก่า
17.         oldSystem.makePayment(String.valueOf(amount));
18.     }
19. }

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         OldPaymentSystem oldSystem = new OldPaymentSystem();
6.         NewPaymentGateway adapter = new
7.             PaymentAdapter(oldSystem);
8.
9.         adapter.processPayment(100.0);
10.    }
11. }

```

ตัวอย่างที่ 2 การเชื่อมต่อ API ของผู้ให้บริการที่แตกต่างกัน ระบบการจองโรงแรมต้องเชื่อมต่อกับ API ของผู้ให้บริการหลายราย (เช่น API A และ API B) แต่แต่ละ API มีโครงสร้างและอินเทอร์เฟซที่แตกต่างกัน การเขียนโค้ดรองรับ API แต่ละตัวอาจซับซ้อนและยุ่งยาก จึงใช้ Adapter Pattern เพื่อสร้างตัวแปลงอินเทอร์เฟซของแต่ละ API ให้เข้ากับอินเทอร์เฟซมาตรฐาน

```

1. // Target Interface
2. interface HotelBooking {
3.     void bookRoom(String hotelName);
4. }

```

```

1. // Adaptee (API A)
2. class APIA {
3.     public void reserve(String hotel) {
4.         System.out.println("Room reserved at " + hotel);
5.     }
6. }

```

```

1. // Adapter สำหรับ API A
2. class APIAAdapter implements HotelBooking {
3.
4.     private APIA apiA;
5.
6.     public APIAAdapter(APIA apiA) {
7.         this.apiA = apiA;
8.     }
9.
10.    @Override
11.    public void bookRoom(String hotelName) {

```

```

12.         apiA.reserve(hotelName); // แปลงการเรียกใช้งาน
13.     }
14.}

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         HotelBooking booking = new APIAAAdapter(new APIA());
6.         // Output: Room reserved at Grand Hotel using API A.
7.         booking.bookRoom("Grand Hotel");
8.     }
9. }

```

ตัวอย่างที่ 3 การรวมระบบสื่อสารระหว่างบริการ (Message Queue) บริษัทต้องรวมระบบส่งข้อความ เช่น RabbitMQ และ Kafka ซึ่งมีอินเทอร์เฟซการใช้งานที่แตกต่างกัน แต่ต้องการให้สามารถทำงานร่วมกันในระบบเดียวได้ ใช้ Adapter Pattern เพื่อแปลงอินเทอร์เฟซของระบบส่งข้อความแต่ละตัวให้รองรับอินเทอร์เฟซมาตรฐานเดียวกัน

Adapter Pattern มีความสำคัญในสถานการณ์ที่ต้องการเชื่อมต่อระบบที่แตกต่างกัน โดยเฉพาะเมื่อ

- ระบบเก่าต้องทำงานร่วมกับระบบใหม่
- การรวมอุปกรณ์หรือบริการที่มีอินเทอร์เฟซแตกต่างกัน
- การเชื่อมต่อ API ของผู้ให้บริการหลายราย

Adapter Pattern ช่วยให้การเชื่อมต่อระบบง่ายขึ้น และลดต้นทุนในการปรับเปลี่ยนระบบ

4.3 Bridge Pattern

Bridge Pattern ช่วยแยกนามธรรม (Abstraction) และการพัฒนา (Implementation) ออกจากกัน เพื่อให้สามารถพัฒนาระบบและปรับเปลี่ยนได้อย่างอิสระ โดยไม่กระทบต่อกัน

:: *Bridge Pattern* ::

แยก Abstraction และ Implementation ออกจากกัน

เพื่อให้สามารถพัฒนาระบบและปรับเปลี่ยนได้อย่างอิสระ โดยไม่กระทบต่อกัน

ตัวอย่างปัญหา

ต้องการแยกชนิดของบ้าน และประเภทของวัสดุที่ใช้สร้างบ้านออกจากกัน เพื่อให้สามารถเพิ่มบ้าน หรือวัสดุใหม่ได้ง่ายโดยไม่กระทบโค้ดเดิม

ตัวอย่าง Main program สร้างบ้านเดี่ยวที่สร้างด้วยคอนกรีต

```
House detachedConcreteHouse = new DetachedHouse (  
    new ConcreteMaterial ()  
);  
detachedConcreteHouse.construct();
```

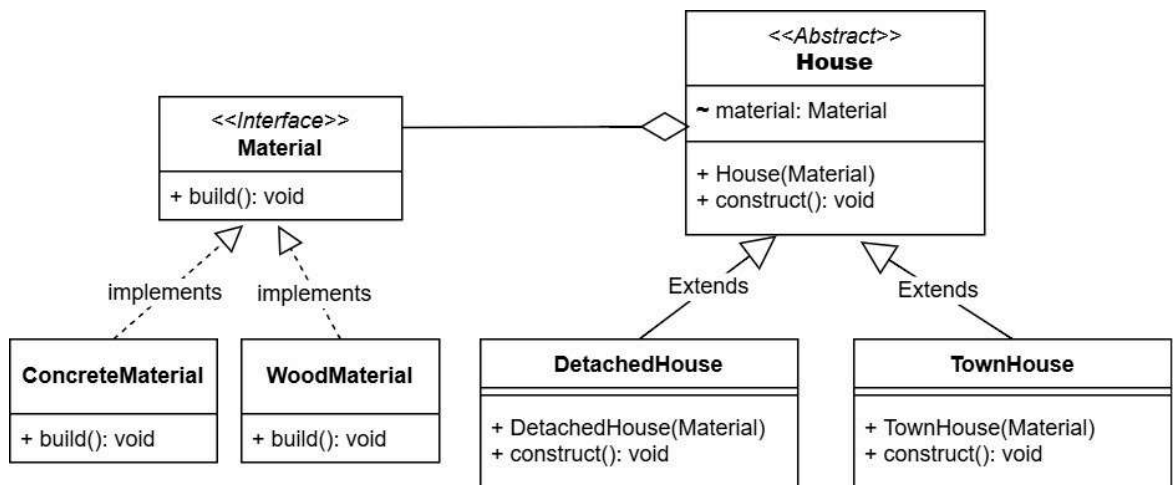
4.3.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Bridge Design Pattern

- จัดการความซับซ้อนของโครงสร้างที่มีหลายมิติ เช่น ถ้ามีหลายชนิดของบ้าน (บ้านเดี่ยว ทาวน์เฮาส์) และหลายประเภทของวัสดุ (คอนกรีต ไม้) การรวมทุกอย่างไว้ในคลาสเดียวจะทำให้โค้ดซับซ้อน
- แยกความรับผิดชอบ (Separation of Concerns) ทำให้ Abstraction (ชนิดของบ้าน) และ Implementation (วัสดุหรือการสร้าง) แยกกัน เพื่อให้สามารถขยายและปรับปรุงได้ง่าย
- เพิ่มความยืดหยุ่นและรองรับการขยาย หากต้องเพิ่มชนิดใหม่ของบ้านหรือวัสดุใหม่ สามารถเพิ่มได้โดยไม่ต้องเปลี่ยนโค้ดเดิม

ตัวอย่างการใช้งาน

ตัวอย่างระบบต้องการสร้างบ้านหลายชนิด เช่น บ้านเดี่ยว และทาวน์เฮาส์ โดยมีวัสดุหลายประเภท เช่น คอนกรีตและไม้ ถ้าต้องการแยกชนิดของบ้าน และประเภทของวัสดุเพื่อให้สามารถเพิ่มบ้านหรือวัสดุใหม่ได้ง่ายโดยไม่กระทบโค้ดเดิม



ภาพที่ xx

Interface Material เป็นอินเทอร์เฟซเพื่อกำหนดการวิธีการสร้างบ้าน (Implementation) เช่น การสร้างบ้านด้วยคอนกรีตหรือการสร้างด้วยไม้

```
1. // Step 1: สร้าง Interface สำหรับการนำไปปฏิบัติ (Implementation)
2. interface Material {
3.     void build();
4. }
```

Concrete Implementation (ConcreteMaterial, WoodMaterial) เป็นการนำวัสดุแต่ละชนิดมาสร้างจริง

```
1. // Step 2: สร้าง Concrete Implementation สำหรับวัสดุแต่ละประเภท
2. class ConcreteMaterial implements Material {
3.
4.     @Override
5.     public void build() {
6.         System.out.println("Building with Concrete.");
7.     }
8. }
```

```

1. class WoodMaterial implements Material {
2.
3.     @Override
4.     public void build() {
5.         System.out.println("Building with Wood.");
6.     }
7. }

```

Abstract Class House เป็นตัวแทนของ Abstraction เช่น ชนิดของบ้าน (บ้านเดี่ยว, ทาวน์เฮาส์) เชื่อมต่อกับ Material ผ่าน Composition

```

1. // Step 3: สร้าง Abstraction สำหรับประเภทของบ้าน
2. abstract class House {
3.
4.     protected Material material; // เชื่อมโยงกับ Material (Bridge)
5.
6.     public House(Material material) {
7.         this.material = material;
8.     }
9.
10.    public abstract void construct();
11.}

```

Concrete Class (DetachedHouse, TownHouse) เป็นการนำ Abstraction แต่ละแบบมาสร้างจริง เช่น บ้านเดี่ยวและทาวน์เฮาส์

```

1. // Step 4: สร้าง Concrete Abstraction สำหรับชนิดของบ้าน
2. class DetachedHouse extends House {
3.
4.     public DetachedHouse(Material material) {
5.         super(material);
6.     }
7.
8.     @Override
9.     public void construct() {
10.        System.out.print("Detached House: ");
11.        material.build();
12.    }
13.}

```

```

1. class TownHouse extends House {
2.     public TownHouse(Material material) {
3.         super(material);
4.     }
5.
6.     @Override

```

```

7.     public void construct() {
8.         System.out.print("Townhouse: ");
9.         material.build();
10.    }
11.}

```

การใช้งานใน Main สามารถจับคู่ชนิดของบ้านและวัสดุได้อย่างอิสระ

```

1. // Step 5: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // บ้านเดี่ยวที่สร้างด้วยคอนกรีต
6.         House detachedConcreteHouse =
7.             new DetachedHouse(new ConcreteMaterial());
8.         // Output: Detached House: Building with Concrete.
9.         detachedConcreteHouse.construct();
10.
11.        // บ้านเดี่ยวที่สร้างด้วยไม้
12.        House detachedWoodHouse =
13.            new DetachedHouse(new WoodMaterial());
14.        // Output: Detached House: Building with Wood.
15.        detachedWoodHouse.construct();
16.
17.        // ทาวน์เฮาส์ที่สร้างด้วยคอนกรีต
18.        House townConcreteHouse =
19.            new TownHouse(new ConcreteMaterial());
20.        // Output: Townhouse: Building with Concrete.
21.        townConcreteHouse.construct();
22.
23.        // ทาวน์เฮาส์ที่สร้างด้วยไม้
24.        House townWoodHouse = new TownHouse(new WoodMaterial());
25.
26.        // Output: Townhouse: Building with Wood.
27.        townWoodHouse.construct();
28.    }
29.}

```

ผลการทำงาน

Detached House: Building with Concrete.

Detached House: Building with Wood.

Townhouse: Building with Concrete.

Townhouse: Building with Wood.

ข้อดีของ Bridge Design Pattern

- แยกความรับผิดชอบ Abstraction และ Implementation ออกจากกัน ทำให้เข้าใจและดูแลรักษาได้ง่าย
- เพิ่มความยืดหยุ่น สามารถเพิ่มชนิดใหม่ของ Abstraction และ Implementation ได้โดยไม่กระทบต่อกัน
- ลดการสร้างคลาสซ้ำซ้อน ไม่ต้องสร้างคลาสใหม่สำหรับการจับคู่ เช่น บ้านเดี่ยว + คอนกรีต, ทาวน์เฮาส์ + ไม้ ฯลฯ

4.3.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Bridge Pattern ถูกนำมาประยุกต์ใช้ในการทำงานจริงได้ ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 1 การพัฒนาระบบบ้านอัจฉริยะ (Smart Home System) ระบบบ้านอัจฉริยะต้องรองรับอุปกรณ์หลายประเภท เช่น หลอดไฟ และ เครื่องปรับอากาศ โดยอุปกรณ์เหล่านี้อาจเชื่อมต่อผ่านเทคโนโลยีที่แตกต่างกัน เช่น Wi-Fi หรือ Bluetooth หากรวมทุกเทคโนโลยีและอุปกรณ์ในคลาสเดียว จะทำให้โค้ดซับซ้อนและยากต่อการบำรุงรักษา สามารถใช้ Bridge Pattern เพื่อแยกชนิดของอุปกรณ์ (Abstraction) และเทคโนโลยีการเชื่อมต่อ (Implementation) ออกจากกัน

```
1. // Step 1: Interface สำหรับการเชื่อมต่อ (Implementation)
2. interface Connection {
3.     void connect();
4. }

1. // Step 2: Concrete Implementation สำหรับเทคโนโลยีต่าง ๆ
2. class WiFiConnection implements Connection {
3.     @Override
4.     public void connect() {
5.         System.out.println("Connecting via Wi-Fi.");
6.     }
7. }

1. class BluetoothConnection implements Connection {
2.     @Override
3.     public void connect() {
4.         System.out.println("Connecting via Bluetooth.");
5.     }
6. }

1. // Step 3: Abstract Class สำหรับอุปกรณ์ (Abstraction)
2. abstract class SmartDevice {
3.
4.     // เชื่อมโยงกับ Connection (Bridge)
```

```

5.     protected Connection connection;
6.
7.     public SmartDevice(Connection connection) {
8.         this.connection = connection;
9.     }
10.
11.     public abstract void operate();
12.}

```

```

1. // Step 4: Concrete Abstraction สำหรับอุปกรณ์แต่ละชนิด
2. class SmartLight extends SmartDevice {
3.
4.     public SmartLight(Connection connection) {
5.         super(connection);
6.     }
7.
8.     @Override
9.     public void operate() {
10.        System.out.print("Smart Light is operating. ");
11.        connection.connect();
12.    }
13.}

```

```

1. class SmartAirConditioner extends SmartDevice {
2.
3.     public SmartAirConditioner(Connection connection) {
4.         super(connection);
5.     }
6.
7.     @Override
8.     public void operate() {
9.         System.out.print("Smart Air Conditioner
10.            is operating. ");
11.        connection.connect();
12.    }
13.}

```

```

1. // Step 5: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // หลอดไฟที่เชื่อมต่อผ่าน Wi-Fi
6.         SmartDevice lightWiFi = new SmartLight(
7.             new WiFiConnection());
8.         // Output: Smart Light is operating.
9.         // Connecting via Wi-Fi.
10.        lightWiFi.operate();
11.
12.        // หลอดไฟที่เชื่อมต่อผ่าน Bluetooth
13.        SmartDevice lightBluetooth = new SmartLight(
14.            new BluetoothConnection());
15.        // Output: Smart Light is operating.
16.        // Connecting via Bluetooth.
17.        lightBluetooth.operate();
18.

```

```

19.         // เครื่องปรับอากาศที่เชื่อมต่อผ่าน Wi-Fi
20.         SmartDevice acWiFi = new SmartAirConditioner(
21.             new WiFiConnection());
22.         // Output: Smart Air Conditioner is operating.
23.         // Connecting via Wi-Fi.
24.         acWiFi.operate();
25.     }
26. }

```

ตัวอย่างที่ 2 การสร้างระบบสั่งซื้อออนไลน์ (E-Commerce Platform) ระบบต้องรองรับการสั่งซื้อหลายประเภท เช่น สินค้าดิจิทัล และสินค้าจริง โดยมีช่องทางการชำระเงินที่หลากหลาย เช่น บัตรเครดิต และ PayPal หากรวมทุกประเภทสินค้า และช่องทางการชำระเงินในคลาสเดียว จะทำให้โค้ดซับซ้อนและเพิ่มจำนวนคลาสที่ไม่จำเป็น

สามารถใช้ Bridge Pattern เพื่อแยกประเภทสินค้า (Abstraction) และช่องทางการชำระเงิน (Implementation) ออกจากกัน

```

1. // Step 1: Interface สำหรับช่องทางการชำระเงิน (Implementation)
2. interface PaymentMethod {
3.     void processPayment(double amount);
4. }

1. // Step 2: Concrete Implementation สำหรับช่องทางต่าง ๆ
1. class CreditCardPayment implements PaymentMethod {
2.     @Override
3.     public void processPayment(double amount) {
4.         System.out.println("Processing credit card payment of
5.             $" + amount);
6.     }
7. }

1. class PayPalPayment implements PaymentMethod {
2.     @Override
3.     public void processPayment(double amount) {
4.         System.out.println("Processing PayPal payment of $"
5.             + amount);
6.     }
7. }

1. // Step 3: Abstract Class สำหรับประเภทสินค้า (Abstraction)
2. abstract class Product {
3.
4.     // เชื่อมโยงกับ PaymentMethod (Bridge)
5.     protected PaymentMethod paymentMethod;
6.
7.     public Product(PaymentMethod paymentMethod) {
8.         this.paymentMethod = paymentMethod;
9.     }

```

```

10.
11.     public abstract void purchase(double amount);
12.}

```

```

1. // Step 4: Concrete Abstraction สำหรับสินค้าประเภทต่าง ๆ
2. class DigitalProduct extends Product {
3.
4.     public DigitalProduct(PaymentMethod paymentMethod) {
5.         super(paymentMethod);
6.     }
7.
8.     @Override
9.     public void purchase(double amount) {
10.         System.out.print("Purchasing digital product. ");
11.         paymentMethod.processPayment(amount);
12.     }
13.}

```

```

1. class PhysicalProduct extends Product {
2.
3.     public PhysicalProduct(PaymentMethod paymentMethod) {
4.         super(paymentMethod);
5.     }
6.
7.     @Override
8.     public void purchase(double amount) {
9.         System.out.print("Purchasing physical product. ");
10.        paymentMethod.processPayment(amount);
11.    }
12.}

```

```

1. // Step 5: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // ซื้อสินค้าดิจิทัลด้วยบัตรเครดิต
6.         Product digitalCredit = new DigitalProduct(
7.             new CreditCardPayment());
8.         // Output: Purchasing digital product.
9.         // Processing credit card payment of $50.0
10.        digitalCredit.purchase(50.0);
11.
12.        // ซื้อสินค้าจริงด้วย PayPal
13.        Product physicalPayPal = new PhysicalProduct(
14.            new PayPalPayment());
15.        // Output: Purchasing physical product.
16.        // Processing PayPal payment of $100.0
17.        physicalPayPal.purchase(100.0);
18.    }
19.}

```

ตัวอย่างที่ 3 การพัฒนาระบบรายงาน (Report Generation System) ระบบต้องรองรับการสร้างรายงานหลายประเภท เช่น รายงานการขาย และรายงานลูกค้า โดยสามารถสร้างรายงานในหลายรูปแบบ เช่น PDF และ Excel หากรวมทุกประเภทของรายงาน และรูปแบบรายงานในคลาสเดียว จะทำให้โค้ดซับซ้อนและไม่ยืดหยุ่น

สามารถใช้ Bridge Pattern เพื่อแยกประเภทของรายงาน (Abstraction) และรูปแบบของรายงาน (Implementation) ออกจากกัน